

Ralph's $\LaTeX$ Macros

Copyright © 2023 Ralph Schleicher*

Version 1.1

Contents

1	Introduction	1
2	Standard $\LaTeX$ Packages	2
2.1	Layout and Formatting	3
2.2	Managing Indices	4
3	Ralph's $\LaTeX$ Macros	5
3.1	Displayed Text	6
3.2	Code and Syntax	6
3.3	Documenting Definitions	9
3.3.1	Definition Environment	10
3.3.2	Functions and Similar Entities	10
3.3.3	Variables and Similar Entities	13
3.3.4	Data Types and Similar Entities	14
3.3.5	$\TeX$ and $\LaTeX$ Entities	15
3.4	Date and Time	16
4	Index	17

1 Introduction

Ralph's $\LaTeX$ macros is a collection of frequently used packages, commands, and environments for writing $\LaTeX$ documents. The first part is a selection of useful standard $\LaTeX$ packages. The second part are Ralph's own $\LaTeX$ commands and environments, which exist either because there is no standard $\LaTeX$ equivalent or as a matter of style.

*Permission is granted to copy, distribute, and/or modify this document under the terms of the GNU Free Documentation License, Version 1.3 or any later version published by the Free Software Foundation; with no Invariant Sections, no Front-Cover Texts, and no Back-Cover Texts.

2 Standard L^AT_EX Packages

The `rspackages` package loads a selection of standard L^AT_EX packages. That is,

fontenc

If the T_EX engine is `latex` or `pdflatex`, enable the T1 font encoding. Otherwise, do nothing.

ifthen

This package provides the `\ifthenelse` command.

alltt

This package provides the `alltt` environment, which is like the `verbatim` environment, except that the escape character `\` and the grouping characters `{` and `}` have their usual meaning.

array

This package provides the `array` and `tabular` environments.

dcolumn

This package provides a system for aligning table column entries at the decimal point.

longtable

This package provides the `longtable` environment for typesetting multi-page tables.

tabularx

This package provides the `tabularx` environment, which is like the `tabular*` environment, but modifies the widths of certain columns, rather than the inter column space, to set a table with the requested total width.

multicol

This package provides the `multicols` environment which allows switching between single-column and multi-column format on the same page.

verbatim

This package provides a new implementation of the `verbatim` environment. It also defines the `\verbatiminput` command to input a whole file verbatim.

ragged2e

This package provides a new implementation of the text alignment environments `center`, `flushleft`, and `flushright`, and the respective `\centering`, `\raggedleft`, and `\raggedright` commands.

enumitem

This package provides a new implementation of the three basic list environments `enumerate`, `itemize`, and `description`.

amsmath

This package provides miscellaneous enhancements for typesetting mathematical formulas. Displayed equations are aligned to the left margin and

equation numbers are placed on the right. For integral symbols, large operator symbols, and named operators, limits are typeset below and above the symbol.

siunitx

This package provides commands for typesetting physical quantities and units in a consistent way.

xcolor

This package provides extensions to the basic facilities of the **color** package.

graphicx

This package provides extensions to the basic facilities of the **graphics** package. It also defines the `\includegraphics` command.

pdfscape

This package provides PDF support for the **landscape** environment.

splitidx

This package provides support for creating multiple indices. The `\newindex` command can be used to create an additional index. The `\index` command is redefined so that you can add an index entry to a dedicated index. The **splitidx** package is loaded with the **protect** option to avoid premature expansion of `\index` arguments.

hyperref

This package provides extensive support for hypertext links in L^AT_EX. Links are neither framed nor coloured.

The **rspackages** package is best loaded directly after the document class, e.g.

```
\documentclass{scrartcl}
\usepackage{rspackages}
```

2.1 Layout and Formatting

All the loaded packages are configured for a consistent style.

\rs@indent

Dimension

The basic indentation for lists and displays. Default is 10 mm.

Nested lists have the same relative indentation. The default label for itemized lists is an en-dash. Description lists don't make any assumption about the formatting of labels.

Sometimes you may want to typeset some content in a slightly smaller font so that the whole thing does not take up too much space. There is a command for this purpose which sets the font size accordingly.

\smallsize

Control Sequence

The font size for small displays. Default is `\footnotesize`.

2.2 Managing Indices

The `rspackages` package calls `\makeindex` and it provides a `\printindex` command that does the right thing, whether or not you use a single index or multiple indices. The `theindex` environment is redefined so that it only prints the actual index in double-column format. That means, you must provide the sectioning command when printing an index in your document. Here is an example for the indices of a programming reference manual.

```
\chapter{Index}
\section*{Concept Index}
\printindex
\section*{Symbol Index}
\printindex[sym]
```

This behaviour of the `theindex` environment gives you full control over how to print an index, whether or not it appears in the table of contents, and what is printed before and after it – without further ado.

An index style file ‘`rspackages.ist`’ is installed along with the `rspackages` package. The recommended shell command to create the index file `<job-name>.ind` is as follows.

```
makeindex -c -s rspackages.ist <job-name>.idx
```

If you use multiple indices, call `splitindex` instead.

```
splitindex -v <job-name>.idx -- -c -s rspackages.ist
```

When the index style file ‘`rspackages.ist`’ is applied, the final index files have the following format.

```
\begin{theindex}
\indexgroup{\indexsymbolsname}
\item \$ (dollar sign)\indexpagesep 1
\indexspace
\indexgroup{\indexnumbersname}
\item 0 (zero)\indexpagesep 1
\indexspace
\indexgroup{A}
\item an entry\indexpagesep 1
\end{theindex}
```

`\indexsymbolsname`

Control Sequence

The name for the symbols group. Default is “Symbols”.

`\indexnumbersname` **Control Sequence**

The name for the numbers group. Default is “Numbers”.

`\indexspace` **Control Sequence**

The space between adjacent groups. Default is `\medskip`.

`\indexgroup <name>` **Control Sequence**

The sectioning command for the group header. Default is `\minisec`.

Argument `<name>` is the header of the group.

`\indexpagesep` **Control Sequence**

The delimiter between the index entry and the page number. Default is an em-space.

The default behaviour of the `theindex` environment is to omit group headers and only insert `\indexspace` between groups. If you redefine the `theindex` environment, you must call `\indexsetup` at the beginning of the index.

```
\RenewDocumentEnvironment{theindex}{}{%  
  % The ‘multicols*’ environment does not balance  
  % the columns on the last page.  
  \begin{multicols*}{2}  
  \indexsetup[headers]{}{%  
    \end{multicols*}}
```

`\indexsetup <options>?` **Control Sequence**

Adjust the typesetting commands and formatting parameters for the index. Must be called locally in a `TEX` group.

Optional argument `<options>` is a comma separated list of options to change the formatting of the index.

headers

Print group headers.

nogroups

Disable grouping, i.e. omit group headers and disable `\indexspace`.

comma

Separate the index entry and the page number by a comma followed by a space.

3 Ralph’s $\text{\LaTeX}$ Macros

Ralph’s $\text{\LaTeX}$ macros can be loaded all at once by using the `rsmacros` package or individually as documented in the respective section.

3.1 Displayed Text

The `rsdisplay` package provides the `display` environment for displayed text.

`display *? <options>?`

Environment

Like the `displaymath` environment but for vertical material. This environment indents the left margin and inserts space above and below the text body.

- The optional star argument omits the indentation. This is a nice alternative for a ‘here’ float.
- Optional argument `<options>` is a comma separated list of options.

`big` Vertical space around the displayed content is a `\bigskipamount`. This is the default.

`medium`

Vertical space around the displayed content is a `\medskipamount`.

`small`

Vertical space around the displayed content is a `\smallskipamount`.

`par` Vertical space around the displayed content is a `\parskip`.

`none` No vertical space around the displayed content.

For example,

```
\begin{display}
Some text to be displayed.
\end{display}
```

is printed as

Some text to be displayed.

Displayed math

$$\sin^2 x + \cos^2 x = 1$$

has the same spacing.

3.2 Code and Syntax

The `rsmetasyntax` package provides macros for indicating commands, symbol names (identifiers), and meta-syntactic variables.

`\code *? <text>`

Control Sequence

Print `<text>` as literal text in a typewriter type font. The special characters

`\ { } # $ % & ~ _ ~`

must be escaped. Two control sequences have another meaning than usual; `\` prints a backslash character and `\_` prints a visible space. The star variant allows line breaks in $\langle text \rangle$. Whitespace is not significant in $\langle text \rangle$ but the active tilde character can be used to insert a no-break space.

As a rule of thumb, single word commands, options, and symbol names should be printed without quotes. Examples on the other hand, should be printed with single quotes.

\symb $\langle name \rangle$ Control Sequence

Print $\langle name \rangle$ as a symbol, i.e. as a command or programming language identifier. The formatting of the `\symb` macro is like `\code` but all characters in $\langle name \rangle$ print itself. For example, `\symb{\symb}` prints `\symb`.

Please note that the grouping characters `{` and `}` have to be nested properly in $\langle name \rangle$ and that multiple consecutive space characters are collapsed into a single space character. Additionally, all leading and trailing space characters are ignored.

Contrary to the standard L^AT_EX `\verb` command, the `\code` and `\symb` command can be used inside an argument of a T_EX control sequence. In a list environment, for example, `\item[\symb{name}]` works as expected.

\codeitem $\langle text \rangle$ Control Sequence

The `\codeitem` command is syntactic sugar for `\item[\code{\langle text \rangle}]`.

\symbitem $\langle name \rangle$ Control Sequence

The `\symbitem` command is syntactic sugar for `\item[\symb{\langle name \rangle}]`.

Sooner or later you want to use placeholders inside of code snippets. The following commands print such meta-syntactic variables.

\var $\langle name \rangle$ Control Sequence

Print $\langle name \rangle$ as a meta-syntactic variable. The formatting of the `\var` macro adopts the style of variables in mathematics. For example, `\var{name}` prints *name*.

\meta $\langle name \rangle$ Control Sequence

Print $\langle name \rangle$ as a meta-syntactic variable delimited by angle brackets. The formatting of the `\meta` macro adopts the style of a non-terminal symbol in Backus–Naur form notation. For example, `\meta{name}` prints $\langle name \rangle$.

Operators are frequently used to describe repeating syntactic elements. The Kleene star is quite commonly used to denote zero or more occurrences. Likewise the Kleene plus for one or more occurrences. In regular expressions the question mark denotes an optional pattern, i.e. zero or one occurrence.

\kleene $\langle symbol \rangle$ Control Sequence

Produce a Kleene operator for marking repetitive elements of a grammar; `\kleene*` and `\kleene+` produce the Kleene star and Kleene plus unary operator respectively. For example, `\meta{expr}\kleene*` prints $\langle expr \rangle^*$, which means zero or more occurrences of $\langle expr \rangle$.

metasyntax

Environment

An environment to describe the syntax of a language, for example, a computer programming language, an application programming interface, a file or document format, or a communication protocol.

Regular text is printed as in the `\code` command and such texts denote terminals. Non-terminals should be formatted with the `\meta` command. A set of meta-symbol commands is available inside the body of the `metasyntax` environment denoting syntax constructs.

<code>\=</code>	Production or definition.	
	<code>\meta{foo}\=\meta{bar}</code>	$\langle foo \rangle \rightarrow \langle bar \rangle$
<code>\,</code>	Sequence or concatenation.	
	<code>\meta{foo}\,\meta{bar}</code>	$\langle foo \rangle \langle bar \rangle$
<code>\ </code>	Alternative or choice.	
	<code>\meta{foo}\ \meta{bar}</code>	$\langle foo \rangle \mid \langle bar \rangle$
<code>*</code>	Repetition, zero or more times (Kleene star).	
	<code>\meta{foo}*</code>	$\langle foo \rangle^*$
<code>\+</code>	Repetition, one or more times (Kleene plus).	
	<code>\meta{foo}\+</code>	$\langle foo \rangle^+$
<code>\?</code>	Optional phrase.	
	<code>\meta{foo}\?</code>	$\langle foo \rangle^?$
<code>\(...\)</code> or <code>\group{...}</code>	Grouping.	
	<code>\(foo\ bar\)</code>	$(foo \mid bar)$
	<code>\group{foo\ bar}</code>	$(foo \mid bar)$
<code>\start</code>	Mark the start rule.	
	<code>\start\meta{foo}</code>	$\triangleright \langle foo \rangle$
<code>\default</code>	Mark the default value.	
	<code>\default{42}</code>	<u>42</u>
<code>\&</code>	Continuation, add it at the beginning of a line.	
	<code>\meta{list}\=\meta{item}</code>	$\langle list \rangle \rightarrow \langle item \rangle$
	<code>\&\ \meta{item}\,\meta{list}</code>	$\mid \langle item \rangle \langle list \rangle$
	Each <code>\&</code> increases the indentation by one unit.	
<code>\\</code> , <code>_</code> , <code>\dots</code>	Backslash, visible space, and ellipses.	

<code>\\</code>	<code>\</code>
<code>_</code>	<code>_</code>
<code>\dots</code>	<code>...</code>

3.3 Documenting Definitions

The `rsdefinition` package provides macros to describe functions, variables, data types, and similar entities in a consistent style. The interface is modeled after the respective Texinfo functionality.

For example,

```
\begin{definition}
\defun{forward-line} &optional \var{n}
Move \var{n} lines forward (backward if \var{n} is negative).
\end{definition}
```

is printed as

<code>forward-line</code> &optional <i>n</i>	Function
Move <i>n</i> lines forward (backward if <i>n</i> is negative).	

Start the definition with a `\def...` command to insert the signature of the entity. The body of the definition, i.e. the documentation, follows on succeeding lines.

The signature line is printed in the `\definitionfont` font. The name of the entity is formatted using the `\symb` command. If the entity has parameters, the entire parameter list is either a balanced group or the rest of the line.

The parameter list has several features to format Lisp lambda lists nicely. Parentheses, (and), are active and print itself with a slightly larger spacing to improve readability. The ampersand & is also active. It reads either a balanced group or everything up to the next space, and prints the entire word (including the ampersand) as a lambda list keyword using the `\lambdalistkeyword` command.

Each definition is assigned to a category. The name of the category is displayed at the right-hand margin of the signature line.

You can insert multiple signatures if you want to document the entities together. For example,

```
\begin{definition}
\defun[double]{sin}(double \var{x})
\defun[double]{cos}(double \var{x})
Return the sine or cosine of \var{x} respectively.
\end{definition}
```

is printed as

<i>double</i> <code>\sin</code> (<i>double</i> x)	Function
<i>double</i> <code>\cos</code> (<i>double</i> x)	Function
Return the sine or cosine of x respectively.	

There are three universal `\def...` commands with an explicit category name and several specific commands with an implicit category name. See table 1 on page 11, for a list of all these commands.

3.3.1 Definition Environment

<code>\definition</code>	Environment
--------------------------	--------------------

An environment for documenting commands and application programming interfaces. Inside a definition environment, use the `\defn`, `\defvr`, or `\deftp` command to insert the signature of a function, variable, or data type like entity respectively. The documentation of the entity is added below the signature line.

<code>\definitionfont</code>	Control Sequence
------------------------------	-------------------------

The main font for the signature line of a definition.

Default is ‘`\normalfont\slshape`’. If the definition takes arguments, mark the parameters with either `\var` or `\meta` so that the documentation can refer them.

<code>\definitioncategoryfont</code>	Control Sequence
--------------------------------------	-------------------------

The font for the category of a definition.

Default is ‘`\normalfont\bfseries\small`’. If the document class is a KOMA-Script class, `\definitioncategoryfont` uses the respective disposition font.

<code>\lambdalistkeyword</code> $\langle word \rangle$	Control Sequence
--	-------------------------

Format $\langle word \rangle$ as a lambda list keyword. For example, ‘`\lambdalistkeyword{\&rest}`’ is printed as `&rest`.

<code>\lambdalistkeywordfont</code>	Control Sequence
-------------------------------------	-------------------------

The font for a lambda list keyword.

Default is ‘`\normalfont\small`’.

3.3.2 Functions and Similar Entities

This section describes the commands for documenting functions and similar entities.

<code>\defn</code> $\langle category \rangle$ $\ast^?$ $\langle data-type \rangle^?$ $\langle symbol-name \rangle$ $\langle lambda-list \rangle$	Control Sequence
--	-------------------------

The `\defn` command is the general definition command for functions or similar entities that may take arguments.

- First argument $\langle category \rangle$ is the category name, for example, “Function” if the entity is a function.

Command	Category	Arguments	Typed
<code>\deffn</code>	<i>explicit</i>	✓	✓
<code>\defun</code>	Function	✓	✓
<code>\defmacro</code>	Macro	✓	✓
<code>\defgeneric</code>	Generic Function	✓	✓
<code>\defmethod</code>	Method	✓	✓
<code>\defspec</code>	Special Form	✓	✓
<code>\defcom</code>	Command	✓	✓
<code>\defvr</code>	<i>explicit</i>		✓
<code>\defvar</code>	Special Variable		✓
<code>\defparam</code>	Parameter		✓
<code>\defconst</code>	Constant		✓
<code>\defopt</code>	User Option		✓
<code>\deftp</code>	<i>explicit</i>		
<code>\deftype</code>	Data Type		
<code>\defstruct</code>	Structure		
<code>\defclass</code>	Class		
<code>\defcondition</code>	Condition		
<code>\deftex</code>	Control Sequence	✓	
<code>\defenv</code>	Environment	✓	
<code>\defcount</code>	Counter		
<code>\defdimen</code>	Dimension		
<code>\defskip</code>	Glue		
<code>\defmuskip</code>	Math Glue		
<code>\defbox</code>	Box Register		
<code>\deftoks</code>	Token Register		
<code>\defif</code>	Conditional		

Table 1: Definition Commands

- Optional second argument $\langle data-type \rangle$ is the data type of the return value of the function.
- Third argument $\langle symbol-name \rangle$ is the name of the function.
- Fourth argument $\langle lambda-list \rangle$ are the parameters of the function. The $\langle lambda-list \rangle$ argument is either a balanced group (in which case it can span multiple lines) or the rest of the line.

If the star variant is specified, $\langle data-type \rangle$ is printed on a line by itself. Otherwise, the data type is put on the same line as the function name and the arguments.

`\defun *?  $\langle data-type \rangle$ ?  $\langle symbol-name \rangle$   $\langle lambda-list \rangle$` **Control Sequence**

The `\defun` command is like the `\defn` command with the implicit category name “Function”. The actual category name is provided by the `\defunname` command.

`\defmacro *?  $\langle data-type \rangle$ ?  $\langle symbol-name \rangle$   $\langle lambda-list \rangle$` **Control Sequence**

The `\defmacro` command is like the `\defn` command with the implicit category name “Macro”. The actual category name is provided by the `\defmacroname` command.

`\defgeneric *?  $\langle data-type \rangle$ ?  $\langle symbol-name \rangle$   $\langle lambda-list \rangle$` **Control Sequence**

The `\defgeneric` command is like the `\defn` command with the implicit category name “Generic Function”. The actual category name is provided by the `\defgenericname` command.

Generic functions are CLOS¹ entities describing a collection of methods with the same symbol name and a similar argument list structure. Other object-oriented programming languages name this concept abstract method or interface. To use another term, say, for example,

```
\renewcommand\defgenericname{Abstract Method}
```

at the beginning of your document.

`\defmethod *?  $\langle data-type \rangle$ ?  $\langle symbol-name \rangle$   $\langle lambda-list \rangle$` **Control Sequence**

The `\defmethod` command is like the `\defn` command with the implicit category name “Method”. The actual category name is provided by the `\defmethodname` command.

`\defspec *?  $\langle data-type \rangle$ ?  $\langle symbol-name \rangle$   $\langle lambda-list \rangle$` **Control Sequence**

The `\defspec` command is like the `\defn` command with the implicit category name “Special Form”. The actual category name is provided by the `\defspecname` command.

A special form is a primitive Lisp expression that looks like a function call, but is treated specially by the Lisp system.

¹Common Lisp Object System

`\defcom *? <data-type>? <symbol-name> <lambda-list>` **Control Sequence**

The `\defcom` command is like the `\deffn` command with the implicit category name “Command”. The actual category name is provided by the `\defcomname` command.

3.3.3 Variables and Similar Entities

This section describes the commands for documenting variables and similar entities.

`\defvr <category> *? <data-type>? <symbol-name>` **Control Sequence**

The `\defvr` command is the general definition command for variables or similar entities.

- First argument `<category>` is the category name, for example, “Variable” if the entity is a variable.
- Optional second argument `<data-type>` is the data type of the variable.
- Third argument `<symbol-name>` is the name of the variable.

If the star variant is specified, `<data-type>` is printed on a line by itself. Otherwise, the data type is put on the same line as the variable name.

For example,

```
\begin{definition}
\defvr{Variable}*[char *]{program_invocation_name}
This variable's value is the name that was used to invoke
the program running in the current process.
\end{definition}
```

is printed as

```
char *
program_invocation_name Variable
    This variable's value is the name that was used to invoke the program running
    in the current process.
```

There are specialized commands for common category names.

`\defvar *? <data-type>? <symbol-name>` **Control Sequence**

The `\defvar` command is like the `\defvr` command with the implicit category name “Special Variable”. The actual category name is provided by the `\defvarname` command.

`\defparam *? <data-type>? <symbol-name>` **Control Sequence**

The `\defparam` command is like the `\defvr` command with the implicit category name “Parameter”. The actual category name is provided by the `\defparamname` command.

`\defconst *? <data-type>? <symbol-name>` **Control Sequence**

The `\defconst` command is like the `\defvr` command with the implicit category name “Constant”. The actual category name is provided by the `\defconstname` command.

`\defopt *? <data-type>? <symbol-name>` **Control Sequence**

The `\defopt` command is like the `\defvr` command with the implicit category name “User Option”. The actual category name is provided by the `\defoptname` command.

3.3.4 Data Types and Similar Entities

This section describes the commands for documenting data types and similar entities.

`\deftp <category> <symbol-name>` **Control Sequence**

The `\deftp` command is the general definition command for data types or similar entities.

- First argument `<category>` is the category name, for example, “Data Type” if the entity is a data type.
- Second argument `<symbol-name>` is the name of the data type.

`\deftype <symbol-name>` **Control Sequence**

The `\deftype` command is like the `\deftp` command with the implicit category name “Data Type”. The actual category name is provided by the `\deftypename` command.

`\defstruct <symbol-name>` **Control Sequence**

The `\defstruct` command is like the `\deftp` command with the implicit category name “Structure”. The actual category name is provided by the `\defstructname` command.

`\defclass <symbol-name>` **Control Sequence**

The `\defclass` command is like the `\deftp` command with the implicit category name “Class”. The actual category name is provided by the `\defclassname` command.

`\defcondition <symbol-name>` **Control Sequence**

The `\defcondition` command is like the `\deftp` command with the implicit category name “Condition”. The actual category name is provided by the `\defconditionname` command.

If your programming language only provides exceptions, say, for example,

```
\renewcommand\defconditionname{Exception}
```

at the beginning of your document.

3.3.5 T_EX and L^AT_EX Entities

There are specialized commands for documenting T_EX and L^AT_EX entities.

`\deftex` *<symbol-name>* *<lambda-list>* **Control Sequence**

The `\deftex` command is like the `\defn` command with the implicit category name “Control Sequence”. The actual category name is provided by the `\deftexname` command.

`\defenv` *<symbol-name>* *<lambda-list>* **Control Sequence**

The `\defenv` command is like the `\defn` command with the implicit category name “Environment”. The actual category name is provided by the `\defenvname` command.

`\defcount` *<symbol-name>* **Control Sequence**

The `\defcount` command is like the `\defvr` command with the implicit category name “Counter”. The actual category name is provided by the `\defcountname` command.

`\defdimen` *<symbol-name>* **Control Sequence**

The `\defdimen` command is like the `\defvr` command with the implicit category name “Dimension”. The actual category name is provided by the `\defdimenname` command.

`\defskip` *<symbol-name>* **Control Sequence**

The `\defskip` command is like the `\defvr` command with the implicit category name “Glue”. The actual category name is provided by the `\defskipname` command.

`\defmuskip` *<symbol-name>* **Control Sequence**

The `\defmuskip` command is like the `\defvr` command with the implicit category name “Math Glue”. The actual category name is provided by the `\defmuskipname` command.

`\defbox` *<symbol-name>* **Control Sequence**

The `\defbox` command is like the `\defvr` command with the implicit category name “Box Register”. The actual category name is provided by the `\defboxname` command.

`\deftoks` *<symbol-name>* **Control Sequence**

The `\deftoks` command is like the `\defvr` command with the implicit category name “Token Register”. The actual category name is provided by the `\deftoksname` command.

`\defif` *<symbol-name>* **Control Sequence**

The `\defif` command is like the `\defvr` command with the implicit category name “Conditional”. The actual category name is provided by the `\defifname` command.

3.4 Date and Time

The `rsdatetime` package provides macros to print the date and time in ISO 8601 format. This is a lightweight solution. If you need more advanced capabilities, use the `datetime2` package.

`\isodate *? <precision>?`

Control Sequence

Insert the current date in extended ISO 8601 format.

- Optional argument `<precision>` specifies the precision. Value is either `year`, `month`, `day`, `hour`, `minute`, or `second`. Default is `day`. If the precision is a time of day unit, the time is separated from the date by a ‘T’ character.

If the star indicator is present, utilize the basic format.

`\isotime *? <precision>?`

Control Sequence

Insert the current time in extended ISO 8601 format.

- Optional argument `<precision>` specifies the precision. Value is either `hour`, `minute`, or `second`. Default is `minute`.

If the star indicator is present, utilize the basic format.

Whether or not a second precision is actually supported depends on the underlying T_EX engine. If not, the second of the minute defaults to zero.

Table 2 on page 16 lists the date and time formats produced by the `\isodate` and `\isotime` commands.

Extended Format		Basic Format	
Command	Result	Command	Result
<code>\isodate</code>	1985-04-12	<code>\isodate*</code>	19850412
<code>\isodate[year]</code>	1985	<code>\isodate*[year]</code>	1985
<code>\isodate[month]</code>	1985-04	<code>\isodate*[month]</code>	198504
<code>\isodate[day]</code>	1985-04-12	<code>\isodate*[day]</code>	19850412
<code>\isodate[hour]</code>	1985-04-12T10	<code>\isodate*[hour]</code>	19850412T10
<code>\isodate[minute]</code>	1985-04-12T10:15	<code>\isodate*[minute]</code>	19850412T1015
<code>\isotime</code>	10:15	<code>\isotime*</code>	1015
<code>\isotime[hour]</code>	10	<code>\isotime*[hour]</code>	10
<code>\isotime[minute]</code>	10:15	<code>\isotime*[minute]</code>	1015

Table 2: Date and Time Formats

4 Index

Concept Index

alignment
 decimal point 2
 equation 2
alltt_{sty} 2
amsmath_{sty} 2
array_{sty} 2
backslash 7f.
color 3
color_{sty} 3
conditional 2
data type definition 14
date 16
dcolumn_{sty} 2
decimal point 2
definition
 data type 14
 function 10
 variable 13
documentation 9
ellipses 8
encoding
 font 2
enumitem_{sty} 2
exception 14
font encoding 2
fontenc_{sty} 2
format
 multi-column 2
function definition 10
grammar 8
graphics_{sty} 3
graphicx_{sty} 3
hyperref_{sty} 3
hypertext link 3
identifier 7
ifthen_{sty} 2
index 4
 style file 4
ISO 8601 16
<job-name>.ind 4
Kleene operator 7
Kleene plus operator 8
Kleene star operator 8
link
 hypertext 3
list
 nested 3
literal text 6
longtable_{sty} 2
makeindex 4
meta-syntactic variable 7
multi-column format 2
multi-page table 2
multicol_{sty} 2
nested list 3
non-terminal symbol 7
number 3
occurrence 7
operator
 Kleene plus 8
 Kleene star 8
pdfscape_{sty} 3
physical quantity 3
quantity 3
ragged2e_{sty} 2
repetition 7
rsdatetime_{sty} 16
rsdefinition_{sty} 9
rsdisplay_{sty} 6
rsmacros_{sty} 5
rsmetasyntax_{sty} 6

<code>rspackages_{sty}</code>	2	<code>repetition</code>	8
<code>rspackages.ist</code>	4	<code>sequence</code>	8
<code>siunitx_{sty}</code>	3	<code>start rule</code>	8
<code>splitidx_{sty}</code>	3	<code>table</code>	
<code>splitindex</code>	4	<code>multi-page</code>	2
<code>style file</code>		<code>tabularx_{sty}</code>	2
<code>index</code>	4	<code>time</code>	16
<code>symbol</code>		<code>unit</code>	3
<code>non-terminal</code>	7	<code>variable</code>	
<code>symbol name</code>	7	<code>meta-syntactic</code>	7
<code>syntax</code>	8	<code>variable definition</code>	13
<code>alternative</code>	8	<code>verbatim_{sty}</code>	2
<code>default value</code>	8	<code>visible space</code>	7f.
<code>group</code>	8	<code>xcolor_{sty}</code>	3
<code>optional</code>	8		
<code>production</code>	8		

Index of L^AT_EX Macros

<code>_</code>	7f.	<code>\defconst</code>	14
<code>\&</code>	8	<code>\defconstname</code>	14
<code>*</code>	8	<code>\defcount</code>	15
<code>\+</code>	8	<code>\defcountname</code>	15
<code>\,</code>	8	<code>\defdimen</code>	15
<code>\=</code>	8	<code>\defdimenname</code>	15
<code>\?</code>	8	<code>\defenv</code>	15
<code>\</code>	7f.	<code>\defenvname</code>	15
<code>alltt_{env}</code>	2	<code>\deffn</code>	10
<code>array_{env}</code>	2	<code>\defgeneric</code>	12
<code>center_{env}</code>	2	<code>\defgenericname</code>	12
<code>\centering</code>	2	<code>\defif</code>	15
<code>\code</code>	6	<code>\defifname</code>	15
<code>\codeitem</code>	7	<code>definition_{env}</code>	10
<code>\default</code>	8	<code>\definitioncategoryfont</code>	10
<code>\defbox</code>	15	<code>\definitionfont</code>	10
<code>\defboxname</code>	15	<code>\defmacro</code>	12
<code>\defclass</code>	14	<code>\defmacroname</code>	12
<code>\defclassname</code>	14	<code>\defmethod</code>	12
<code>\defcom</code>	12	<code>\defmethodname</code>	12
<code>\defcomname</code>	12	<code>\defmuskip</code>	15
<code>\defcondition</code>	14	<code>\defmuskipname</code>	15
<code>\defconditionname</code>	14	<code>\defopt</code>	14
		<code>\defoptname</code>	14

<code>\defparam</code>	13	<code>\indexsetup</code>	5
<code>\defparamname</code>	13	<code>\indexspace</code>	5
<code>\defskip</code>	15	<code>\indexsymbolsname</code>	4
<code>\defskipname</code>	15	<code>\isodate</code>	16
<code>\defspec</code>	12	<code>\isotime</code>	16
<code>\defspecname</code>	12	<code>itemize_{env}</code>	2
<code>\defstruct</code>	14	<code>\kleene</code>	7
<code>\defstructname</code>	14	<code>\lambdalistkeyword</code>	10
<code>\deftex</code>	15	<code>\lambdalistkeywordfont</code>	10
<code>\deftexname</code>	15	<code>landscape_{env}</code>	3
<code>\deftoks</code>	15	<code>longtable_{env}</code>	2
<code>\deftoksname</code>	15	<code>\makeindex</code>	4
<code>\deftp</code>	14	<code>\meta</code>	7
<code>\deftype</code>	14	<code>metasyntax_{env}</code>	8
<code>\deftypename</code>	14	<code>multicols_{env}</code>	2
<code>\defun</code>	12	<code>\newindex</code>	3
<code>\defunname</code>	12	<code>\printindex</code>	4
<code>\defvar</code>	13	<code>\raggedleft</code>	2
<code>\defvarname</code>	13	<code>\raggedright</code>	2
<code>\defvr</code>	13	<code>\smallsize</code>	3
<code>description_{env}</code>	2	<code>\start</code>	8
<code>display_{env}</code>	6	<code>\symb</code>	7
<code>\dots</code>	8	<code>\symbitem</code>	7
<code>enumerate_{env}</code>	2	<code>tabular_{env}</code>	2
<code>flushleft_{env}</code>	2	<code>tabular*_{env}</code>	2
<code>flushright_{env}</code>	2	<code>tabularx_{env}</code>	2
<code>\group</code>	8	<code>theindex_{env}</code>	4
<code>\ifthenelse</code>	2	<code>\var</code>	7
<code>\includegraphics</code>	3	<code>\verb</code>	7
<code>\index</code>	3	<code>verbatim_{env}</code>	2
<code>\indexgroup</code>	5	<code>\verbatiminput</code>	2
<code>\indexnumbersname</code>	4		
<code>\indexpagesep</code>	5		